

Image Analysis and Object Classification

Using Classical Computer Vision

Student Project Report

VITyarthi - Computer Vision Project Report

2. Introduction

This project is a Computer Vision pipeline that identifies and classifies geometric shapes (circles, squares, triangles) from noisy images. It uses OpenCV to process the images and extract numerical features, which are then passed to a classifier.

3. Problem Statement

The goal of this project is to build a system that can identify shapes using classical computer vision techniques rather than deep learning. It demonstrates how image filtering, contour extraction, and geometric feature calculation can be combined with a K-Nearest Neighbors (KNN) classifier.

4. Functional Requirements

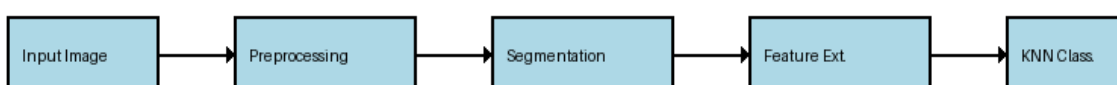
- Generate a dataset of 600 synthetic shape images.
- Process images using grayscale conversion and Gaussian blur.
- Segment shapes using Otsu's thresholding.
- Extract 13 numerical shape features.
- Classify shapes using a K=5 KNN algorithm.
- Provide a command-line interface for training and testing.

5. Non-functional Requirements

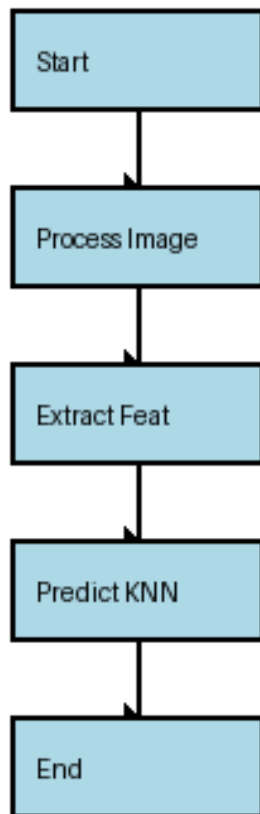
- Performance: The system should run on a standard CPU without requiring GPU acceleration.
- Maintainability: The code is split into distinct modules for preprocessing, feature extraction, and classification.
- Reproducibility: The dataset is generated locally via a script so anyone can reproduce the environment.
- Usability: The terminal interface uses clear arguments for all operations.

6. System Architecture & 7. Design Diagrams

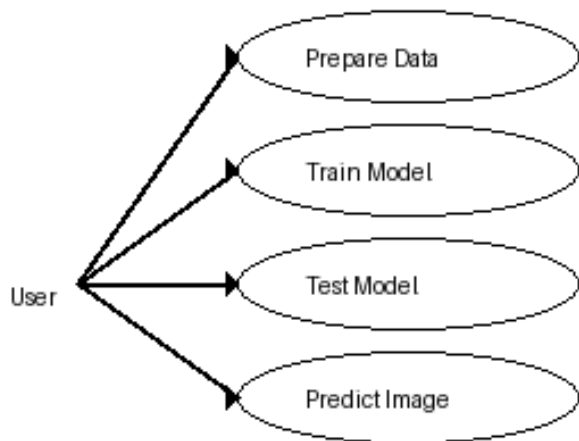
The pipeline consists of Preprocessing, Segmentation, Feature Extraction, and Classification modules. Below are the design diagrams.



VITyarthi - Computer Vision Project Report



VITyarthi - Computer Vision Project Report



ShapeClassifier
+ k: int + X_train + y_train + classes
+ train() + predict() + save() + load()

8. Design Decisions & Rationale

A synthetic dataset was used because it guarantees that the images will test the specific noise removal and thresholding logic written for this project. A custom KNN classifier was implemented using NumPy so that the classification step could be kept simple and directly show how distance-based classification works. The 13 selected features include Hu Moments to help account for the random rotation of the shapes.

9. Implementation Details

The pipeline executes in this order:

1. Image Loading: The image is loaded via OpenCV.
2. Grayscale Conversion: Color channels are removed.

VITyarthi - Computer Vision Project Report

3. Gaussian Blur: A blur is applied to reduce the synthetic noise.
4. Thresholding: Otsu's method isolates the darker shape from the lighter background.
5. Morphological Processing: Closing and opening operations fill small holes and remove artifacts.
6. Contour Extraction: The largest external contour is selected.
7. Feature Extraction: The code calculates area, perimeter, circularity, aspect ratio, extent, solidity, and 7 Hu Moments.
8. KNN Prediction: The 13 features are compared against the training set using Euclidean distance (K=5).

10. Screenshots / Results

The model achieved 60.00% accuracy on the test set. The confusion matrix shows that circles and squares were frequently confused with each other. This indicates that the current feature set and segmentation process still have room for improvement. Below is the actual confusion matrix.

		Predicted		
		circle	square	triangle
Actual	circle	21	16	6
	square	13	18	5
	triangle	1	3	27

11. Testing Approach

The code is tested using pytest. The tests use simulated dummy inputs (like an empty image with a single drawn circle) to verify that the preprocessing, segmentation, feature extraction, and classification modules execute mathematically without crashing.

VITyarthi - Computer Vision Project Report

12. Challenges Faced

One issue was that the Gaussian noise sometimes caused the contour detector to find extra, incorrect boundaries in the background. Morphological operations were adjusted to reduce these extra detections, and a filter was added to select only the largest contour. Additionally, keeping the KNN features balanced was a challenge because the raw area feature is much larger than the Hu Moments.

13. Learnings & Key Takeaways

One thing I learned from the project was that segmentation directly affects the final classifier result. If the contour is incorrect due to remaining noise, the extracted features like area and perimeter are also incorrect, causing the KNN to fail.

14. Future Enhancements

The most necessary enhancement is feature normalization. Because Euclidean distance is used, large numbers like contour area dominate the smaller Hu Moments. Standardizing the features before KNN prediction would likely improve the 60.00% accuracy significantly.

15. References

1. OpenCV Documentation: <https://docs.opencv.org/>
2. Python NumPy Documentation: <https://numpy.org/doc/>